Seminar 03

30.10.2026

Lernziel: JavaScript-Grundkonzepte testbasiert erarbeiten: implementieren und prüfen.

1. Ablauf und Setup

Erarbeitet heute die Grundkonzepte von JavaScript testbasiert an einem Notenrechner („GradeCalc“):

1. Öffnet `index.html` im Browser: unten seht ihr eine Liste mit roten und grünen Tests
2. Implementiert die Funktionen in `seminar.js`, bis alle Tests grün sind

Setup, Drei Dateien:

- `index.html` (Startseite, lädt alles)
- `seminar.js` (eure Implementierung)
- `seminar-tests.js` (die Tests, **nicht verändern**).

Vorlesungsfolien, MDN (developer.mozilla.org) und die Browser-Konsole (F12) sind eure Werkzeuge.

2. Typen und Coercion

Ohne die Ausdrücke auszuführen: Tragt eure Vorhersagen direkt auf der Testseite (`index.html`) ein – bei Vergleichen wählt ihr `true` oder `false`, bei den übrigen Ausdrücken tippt ihr euer Ergebnis:

```javascript
"5" + 3      // Wert?
"5" - 3      // Wert?
5 === "5"    // true oder false?
5 == "5"     // …
// außerdem: 0 == false, "" == 0, [] == false,
//           null == 0, null >= 0, typeof null
```

3. Variablen und Hoisting

3.1. MAX_POINTS

Legt `MAX_POINTS` so an, dass der Test „nicht neu zuweisbar“ besteht (`100`).

3.2. Hoisting-Vorhersage

Wählt auf der Testseite (`index.html`) für jede dieser Zeilen, was sie loggt – `undefined` oder `ReferenceError`:

```javascript
console.log(a);   // var a = 1;
console.log(b);   // let b = 2;
```

3.3. Call Stack bis zum Anschlag

Implementiert `stackOverflow()` so, dass sie einen `RangeError` (`Maximum call stack size exceeded`) auslöst, mit Rekursion.

4. Funktionen als First-Class Citizens

Alle Berechnungen erfolgen auf `const grades = [82, 95, 58, 77, 90];` (Punkte, Maximum `MAX_POINTS`).

4.1. Prozente und Bestehen

```JavaScript
function toPercent(grade) { /* TODO */ }
function isPassing(percent) { /* TODO */ } // true ab 50
```

Achtet auf die Grenzfälle im Test: `isPassing(50)` muss `true` liefern, `isPassing(49.9)` muss `false` liefern.

4.2. Ohne Schleifen

Implementiert drei Funktionen, mit **genau einer Array-Methode pro Funktion**, ohne `for`/`while` (der Test prüft den Quelltext!):

```JavaScript
function allPercents(values) { /* TODO: map */ }
function passedPercents(values) { /* TODO: filter */ }
function averagePercent(values) { /* TODO: reduce, ein einziger Wert! */ }
```

4.3. Funktionen als Werte

Die eingebene Funktion fn sollte auf jeden Wert in `values` angewendet werden.

```JavaScript
function applyToAll(values, fn) { /* TODO */ }
```

5. Objekte und Prototypen

5.1. Zwei Wege, ein Ziel

Ein Student mit `getName()`, **zwei Mal auf zwei Wegen**:

```JavaScript
class Student {
    // TODO: constructor(name), getName()
}

const studentProto = {
    // TODO: getName() { ... }
};

function createStudent(name) {
    // TODO: Object.create(...) verwenden
}
```

5.2. Prototypenkette nachweisen

Implementiert `hasOwnGetName(student)`:

- `true`, wenn das Objekt die Methode **selbst** besitzt
- `false`, wenn sie aus der Prototypenkette kommt (Stichwort: `Object.hasOwn`)

5.3. Methoden im Nachhinein

Implementiert `addGreetToProto()`: Sie fügt über `studentProto` **nachträglich** eine Methode `greet()` hinzu, die `"Hallo, <name>"` zurückgibt. Der Test greift danach auf einen **bereits erzeugten** Studenten zu.

6. Event Loop und Asynchronität

6.1. Callbacks vs. Promises

```JavaScript
function loadGradesCallback(callback) { /* TODO: setTimeout, 1000ms */ }
function loadGrades() {
    const gradesFromServer = [82, 95, 58, 77, 90];
    // TODO: new Promise(...) mit setTimeout
}
```

6.2. Ausgabereihenfolge vorhersagen

Der Test führt exakt diesen Code aus. Eure Vorhersage trefft ihr diesmal **ohne Code**: Sortiert auf der Testseite (`index.html`) die fünf Karten in die Reihenfolge, in der die Zeilen ausgeführt werden (Call Stack → Microtasks → Task Queue) – per Drag & Drop oder Klick:

```JavaScript
const order = [];
order.push("S");
Promise.resolve().then(() => order.push("P"));
setTimeout(() => order.push("T"), 0);
Promise.resolve().then(() => order.push("Micro"));
order.push("K");
```

Die Sortierung wird live geprüft – das Ergebnis erscheint direkt unter den Karten, sobald eine Karte bewegt wird.

6.3. Fehler bewusst erleben

Implementiert `triggerUnhandledRejection()`: gibt ein abgelehntes Promise zurück, das nirgends abgefangen wird.

6.4. async/await

Implementiert loadGradesAsync() so, dass sie mit await auf das Ergebnis von loadGrades() aus 6.1 wartet:

```JavaScript
async function loadGradesAsync() {
    // TODO: try/catch, Erfolg: Noten-Array, Fehler: leeres Array []
}
```